Відокремлений структурний підрозділ

«Фаховий коледж інформаційних технологій»

Національного університету «Львівська політехніка»



З лабораторної роботи

з дисципліни «Тестування програмних систем та комплексів»

Виконав:

Студент групи КН-43

Гарбіч Богдан Назарович

Перевірив:

викладач

Роман Скалоцький

Львів 2026

Звіт про виконання лабораторної роботи №3

Тема: Розробка та тестування програмного забезпечення за методологіями TDD (Test-Driven Development) та BDD (Behavior-Driven Development) за допомогою `pytest` та `pytest-bdd`

Репозиторій: `mrdeferpkk/college-testing`

1. Мета роботи

- Навчитися застосовувати підходи **TDD** та **BDD** під час розробки Python-додатків.
- Опанувати роботу з фреймворком `pytest` для написання юніт-тестів та перевірки винятків (exceptions).
- Освоїти синтаксис Gherkin та інтеграцію BDD-сценаріїв за допомогою плагіна `pytest-bdd`.
- Налаштувати автоматизоване середовище тестування та інтеграцію проєкту з системою контролю версій Git / GitHub.

2. Структура проєкту

Проєкт організовано відповідно до стандартних практик розробки на Python:

```
lab_assert_validation/
├── src/                                # Сирцевий код (логіка додатка)
│                                           ├──              __init__.py
│   ├── bank_account.py                  # Клас управління банківським рахунком
│    ├── temperature_converter.py            # Конвертер температур (Цельсій/Фаренгейт)
│   ├── user_registration_validator.py  # Валідація даних реєстрації користувача
│   └── discount_calculator.py          # Розрахунок знижок на товари
├── tests/                                  # Набори автоматизованих тестів
│   ├── test_bank_account.py             # Юніт-тести для BankAccount
│       ├──  test_temperature_converter.py       #  Юніт-тести  для TemperatureConverter
│    ├── test_user_registration.py          # BDD-тести реєстрації користувача
```

```
│   └── test_discount_calculator.py     # BDD-тести розрахунку знижок
├── features/                                # Gherkin-сценарії (BDD)
│     ├── user_registration.feature          # Сценарії перевірки реєстраційних даних
│   └── discount.feature              # Сценарії перевірки розрахунку знижки
├── pytest.ini                            # Конфігурація шляхів для pytest
└── requirements-dev.txt                  # Залежності проєкту (pytest, pytest-bdd)
```

3. Реалізовані модулі та тестування

3.1. Модуль `BankAccount` (TDD)

- **Функціонал:** Реалізовано методи створення рахунку (`__init__`), поповнення (`deposit`), зняття коштів (`withdraw`) та отримання балансу (`get_balance`).
- **Покриття тестами:** Перевірено початковий баланс за замовчуванням та із заданим значенням, некоректні від'ємні суми, а також спроби зняти більше коштів, ніж є на рахунку (з викиданням `ValueError`).

3.2. Модуль `TemperatureConverter` (TDD)

- **Функціонал:** Конвертація температур з Цельсія у Фаренгейт та навпаки.
- **Покриття тестами:** Перевірено критичні точки (0°C, 100°C), обробку параметризованих некоректних типів даних (`TypeError`) та температури нижче абсолютного нуля (-273.15°C / -459.67°F).

3.3. Модуль `UserRegistrationValidator` (BDD)

- **Функціонал:** Валідація email, пароля (мінімум 8 символів та наявність цифри) та віку користувача (18–120 років).
- **BDD-сценарії (`features/user_registration.feature`):** Описано та реалізовано 4 сценарії (`Given / When / Then`), що охоплюють успішну реєстрацію та відхилення за некоректний email, закороткий пароль або недосягнення 18 років.

3.4. Модуль `DiscountCalculator` (BDD)

- **Функціонал:** Розрахунок підсумкової вартості товару з урахуванням відсотка знижки (0–100%).

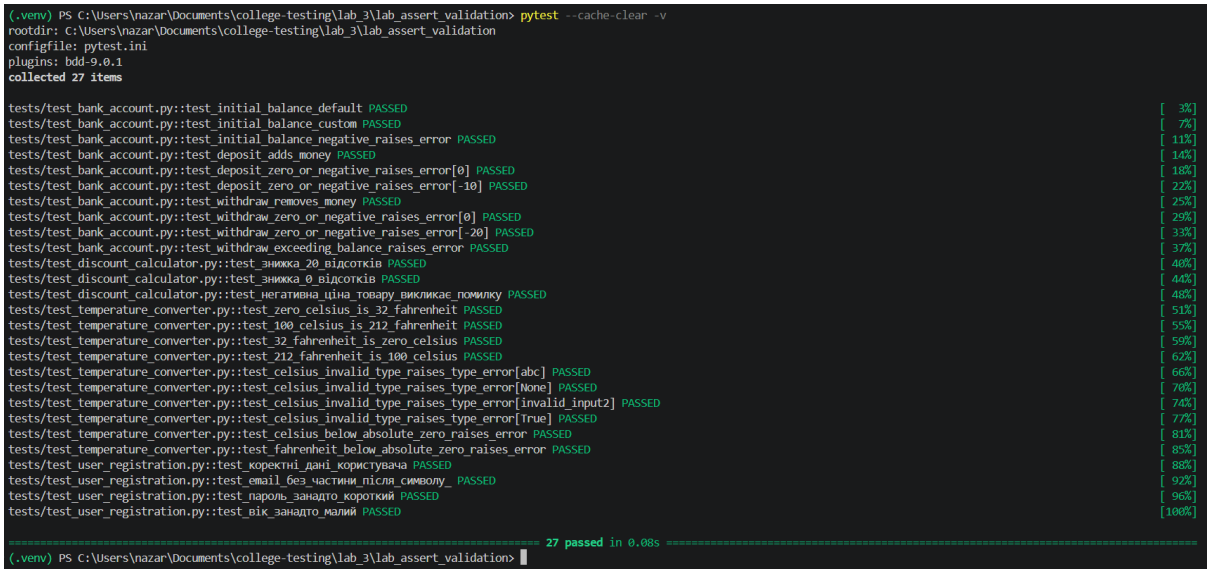
- **BDD-сценарії (`features/discount.feature`):** Описано сценарії обчислення знижки 20%, знижки 0%, а також перевірку реакції на від'ємну ціну товару.

4. Результати виконання тестів

Запуск усіх тестових наборів виконувався в ізольованому віртуальному середовищі (`.venv`) за допомогою команди:

```
pytest --cache-clear -v
```

Підсумок проходження тестів:



- **Загальна кількість тестів:** 27
- **Успішно пройдено (`PASSED`):** 27 (100%)
- **Час виконання:** ~0.09 секунди

Все охоплення код-бази пройшло без помилок, збоїв чи зауважень від тестового фреймворку.

5. Розв'язання проблем у процесі розробки

1. **Шляхи та кодування у Windows:** Початкові проблеми з імпортом розв'язано перенесенням проєкту в директорію без кириличних символів у назві.
2. **Налаштування імпорту:** Додано конфігураційний файл `pytest.ini` з параметром `pythonpath = .` для коректного виявлення модулів із папки `src/`.
3. **Узгодження регістру літер:** Виправлено помилки імпорту класу `DiscountCalculator` та `UserRegistrationValidator` шляхом узгодження CamelCase в оголошенні та тестуванні.

4. **Синхронізація з Git:** Усунуто конфлікти злиття (`fetch first` та `unmerged paths`) при публікації проєкту у віддаленому репозиторії `mrdeferpkk/college-testing`.

6. Висновки

Під час виконання лабораторної роботи було успішно освоєно практики TDD та BDD. Написання тестів перед логікою та описом сценаріїв мовою Gherkin дозволило створити надійний, легко розширюваний та валідований код без критичних помилок. Проєкт повністю протестований (27/27 пройшли) та збережений у віддаленому репозиторії.